King Saud University
College of Computer and Information Sciences
Department of Information Technology

Crime Count Forecasting per District

IT462

Student Name	IDs	Email
Raghad Almutairi	443200793	443200793@student.ksu.edu.sa
Sarah Alruwayte	444200758	444200758@student.ksu.edu.sa
Norah Alfaheed	444200779	444200779@student.ksu.edu.sa
Atheer Budie	444200894	444200894@student.ksu.edu.sa

Section: 2766 | Group: 2
Supervised by: Dr. AFSHAN JAFRI
Date of submission: May 10, 2026

Table of Contents

1. Introduction *4*

1.1 background 4

1.2 problem statement 4

1.3 objectives 4

1.4 scope and limitations 4

2. Data selection and understanding *5*

2.1 dataset description 5

2.2 structure and schema 5

2.3 data quality and initial observations 6

2.4 ethical and legal considerations 6

3. Data preprocessing *6*

3.1 data cleaning 6

3.2 data integration 7

3.3 data reduction 7

3.4 data transformation 8

3.5 summary of preprocessing pipeline 9

4. Rdd operations *9*

4.1 rdd design 10

4.2 key transformations and actions 10

District crime volume aggregation 10

Per-district arrest rate enrichment 11

4.3 reflection on rdd usage 11

5. Sql operations *12*

5.1 dataframe and view creation 12

5.2 key analytical queries 12

Q1: what is the total and average weekly crime count per month across all districts? 12

Q2 : what are the 10 highest crime weeks on record and which district had them? 13

Q3: which districts have an arrest rate above the city-wide average? 13

Q5: how did total annual crime change per district between 2021 and 2024? 14

Q8: which districts consistently exceed the city-wide weekly crime average? 15

5.3 summary of insights from sql 16

6. Machine learning *16*

6.1 problem formulation 16

6.2 feature pipeline 17

6.3 model choice 17

6.4 training and evaluation setup 18
6.5 results 18
6.6 interpretation 18
Feature importance 18
Model interpretation 19
Error analysis 19
6.7 limitations 19
7. Conclusion and future work *20*
7.1 summary of contributions 20
7.2 reflection 20
7.3 future work 20
8. References *21*

List of Tables

Table 1: dataset summary 5
Table 2: dataset schema for key attributes 5
Table 3: cleaning summary 7
Table 4: reduction impact 8
Table 5: preprocessing pipeline summary 9
Table 6: rdd operations summary 9
Table 7: sql queries summary 12
Table 8: ml problem formulation 16
Table 9: full model comparison across all configurations 18
Table 10: reduced rf feature importances 18

List of Figures

Figure 1: spark shell output showing the inferred schema 7
Figure 2: preprocessed dataset sample rows 8
Figure 3: full model comparison 17

1. Introduction

1.1 Background

Urban crime management is a persistent challenge for city governments worldwide. Chicago records hundreds of thousands of incidents annually across 23 active police districts. Despite large-scale historical data availability, this data is used descriptively rather than predictively, limiting police departments' ability to pre-position resources efficiently[1] .

1.2 Problem Statement

This project builds a regression-based forecasting system predicting total weekly crime incidents per Chicago police district using 2021–2024 historical records. Rather than binary risk classification, the system produces quantitative district-level forecasts—distinguishing, for example, between 50 and 200 crimes in a week—enabling precise staffing and patrol decisions[2] . This moves the Chicago Police Department beyond traditional risk mapping toward evidence-based resource allocation.

1.3 Objectives

- To clean and preprocess 971,865 raw crime incident records from the City of Chicago Data Portal into a structured weekly time-series format suitable for machine learning.
- To build and evaluate regression models using Spark MLlib to forecast weekly crime counts per district, comparing multiple feature configurations and algorithms
- To interpret model results, identify the most influential forecasting features, and honestly assess model limitations.

1.4 Scope and Limitations

The project models district-level crime trends using the Chicago Crimes dataset (2021–2024)[3] at weekly granularity across 23 districts, implemented in Apache Spark with Scala[4][5]. Three algorithms are evaluated: Linear Regression, Decision Tree, and Random Forest using RMSE, MAE, and R^2 metrics.

Out of scope: real-time deployment, per-crime-type forecasting, and external variables such as weather or economic data.

The system relies exclusively on historical crime counts, omitting socio-economic or environmental drivers. Weekly district-level aggregation supports strategic planning but lacks the granularity required for block-level tactical interventions.

2. Data Selection and Understanding

2.1 Dataset Description

Table 1: Dataset summary

Attribute	Value
Dataset Name	Crimes – 2001 to Present
Source	City of Chicago Data Portal[3].
Domain	Public Safety / Urban Analytics
Time coverage	2021–2024
Total Records	971,865 incident-level records
Number of Attributes	22 original columns
Number of Files	1 primary flat file
Table Role	Denormalized Fact Table (Captures individual crime events)

2.2 Structure and Schema

The dataset contains 22 attributes spanning four types: numeric, categorical, temporal, and text. The district column serves as the primary spatial key linking all district-level analyses. The id column is the unique incident identifier with zero duplicates confirmed across all 971,865 records.

Table 2: Dataset schema for key attributes

Attribute	Type	Description	Value
id	Numeric	Unique incident identifier	12419690
district	Numeric	Police district number (primary key)	1–31
beat	Numeric	Police beat number	1–2535
latitude	Numeric	GPS latitude coordinate	41.88
longitude	Numeric	GPS longitude coordinate	-87.63
primary_type	Categorical	Main crime category	THEFT, BATTERY
arrest	Categorical	Whether arrest was made	True / False
domestic	Categorical	Whether incident was domestic	True / False
location_description	Categorical	Type of location	STREET, RESIDENCE
date	Temporal	Incident occurrence timestamp	01/01/2021 12:00 AM

Attribute	Type	Description	Value
year	Temporal	Year of incident	2021, 2022, 2023, 2024
updated_on	Temporal	Last record update timestamp	02/08/2021 03:50 PM
case_number	Text	CPD case reference number	JA123456
block	Text	Partially anonymized address	001XX N STATE ST

Note: Primary key is id (unique per incident). Natural key for spatial analysis is district. No foreign keys apply as this is a single-file dataset.

2.3 Data Quality and Initial Observations

Missing values are concentrated in geographic attributes: latitude, longitude, x_coordinate, y_coordinate, and location each had 15,316 nulls; location_description had 4,949; community_area had 79; ward had 24. Key forecasting attributes—district, primary_type, and date—were fully populated.

Temporal checks confirmed zero records outside 2021–2024, zero duplicates, and all 23 district IDs within the valid range of 1–31.

2.4 Ethical and Legal Considerations

The dataset is publicly available under an open government data license permitting educational use. It contains no personally identifiable information: names and full addresses are absent, block is partially anonymized, case numbers cannot link to individuals without restricted records, and coordinates are approximate. The dataset is fully anonymized and legally acceptable for academic use.

3. Data Preprocessing

The preprocessing pipeline transformed 971,865 raw incident records into 4,644 clean, aggregated weekly rows ready for machine learning, covering four components: cleaning, integration, reduction, and transformation.

3.1 Data Cleaning

Missing values were concentrated in geographic columns — latitude, longitude, x_coordinate, y_coordinate, and location each had 15,316 nulls. A bias analysis confirmed no systematic concentration across districts (1.3%–3.57%), crime types (dominant types below 2% missing), or years (downward trend from 3.65% in 2021 to 0.96% in 2024). All rows with missing geographic fields were dropped. No duplicate IDs, invalid dates, or invalid district values were found.

Table 3: Cleaning summary

Issue	Before	After
Records with missing geographic fields	19,302 rows affected	0, all removed
Missing values in latitude / longitude / location	15,316 nulls each	0
Missing values in location_description	4,949 nulls	0
Duplicate incident IDs	0	0, no action needed
Invalid date values	0	0, all dates valid
Out-of-range district values	0	0, all within 1–31
Total records	971,865	952,563

3.2 Data Integration

The dataset is a single CSV file containing all 971,865 records. No multi-source integration or join operations were required.

The schema was inferred by Spark using inferSchema and covers five categories: temporal (date, year, updated_on), spatial (lat/lon, x/y coordinates), categorical (primary_type, location_description, fbi_code, iucr), administrative (district, ward, community_area, beat), and binary (arrest, domestic). Identifier columns (id, case_number, block, location) were later removed during feature selection.

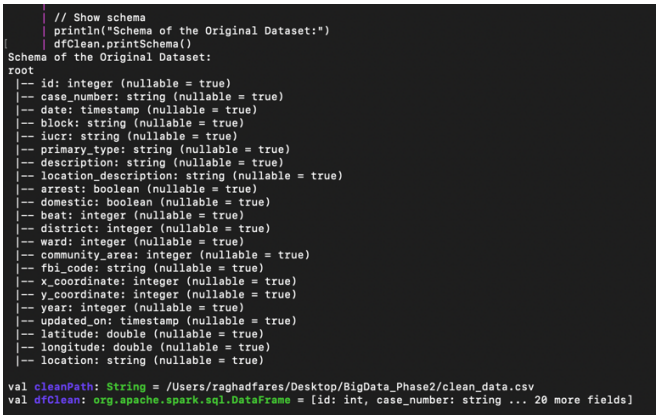

Figure 1: Spark shell output showing the inferred schema

3.3 Data Reduction

Sampling was deliberately not applied, as removing records would distort weekly crime trends across 23 districts and 209 weeks. The full 952,563-record dataset was retained. Feature selection reduced 22 columns to 3 (date, district, primary_type), dropping 19 administrative attributes with no predictive value.

Table 4: Reduction impact

Stage	Rows	Columns
Original raw dataset	971,865	22
After cleaning	952,563	22
After feature selection	952,563	3
After pivoted aggregation	4,644	33

3.4 Data Transformation

Type Conversions: week_start was derived via date_trunc; month and week_no were extracted as integers, converting timestamps into structured temporal features. Scaling and Normalization: Scaling of prev_week_total, month, and week_no was deferred to the ML phase to prevent data leakage; StandardScaler was fitted on training data only.

Feature Engineering: Four new domain-motivated features were created:

New Feature	Type	Domain Motivation
month	Temporal	Captures seasonal crime cycles — crime peaks in July and drops in February as confirmed in SQL analysis
week_no	Temporal	Captures intra-year weekly patterns and holiday effects within each calendar year
district_indexed	Spatial	Encodes each district as a unique category, enabling the model to learn district-specific crime baselines
prev_week_total	Lag	Captures historical momentum — last week's crime count is the strongest available predictor of this week's count

Following all transformation steps, the final preprocessed dataset contains 4,644 rows and 38 columns, as shown in Figure 2 which displays a representative sample of the output.

```
scala> // Selecting a balanced mix of spatial, temporal, behavioral, and engineered features
     | val finalSnapshot = dfFinal.select(
     |   "district",
     |   "week_start",
     |   "month",
     |   "week_no",
     |   "THEFT",
     |   "BATTERY",
     |   "NARCOTICS",
     |   "total_crime_count",
     |   "prev_week_total"
     | )
     |
     | println("Final Preprocessed Dataset Snapshot (15 rows):")
     | finalSnapshot.show(15, truncate = false)
Final Preprocessed Dataset Snapshot (15 rows):
+--------+-------------------+-----+-------+------+-------+---------+-----------------+----------------+
|district|week_start         |month|week_no|THEFT|BATTERY|NARCOTICS|total_crime_count|prev_week_total|
+--------+-------------------+-----+-------+------+-------+---------+-----------------+----------------+
|1       |2021-01-04 00:00:00|1    |1      |39    |17     |3        |123              |47              |
|1       |2021-01-11 00:00:00|1    |2      |39    |18     |0        |124              |123             |
|1       |2021-01-18 00:00:00|1    |3      |39    |13     |2        |118              |124             |
|1       |2021-01-25 00:00:00|1    |4      |21    |18     |0        |96               |118             |
|1       |2021-02-01 00:00:00|2    |5      |27    |13     |1        |104              |96              |
|1       |2021-02-08 00:00:00|2    |6      |28    |25     |1        |100              |104             |
|1       |2021-02-15 00:00:00|2    |7      |29    |17     |0        |104              |100             |
|1       |2021-02-22 00:00:00|2    |8      |40    |16     |0        |133              |104             |
|1       |2021-03-01 00:00:00|3    |9      |26    |30     |1        |127              |133             |
|1       |2021-03-08 00:00:00|3    |10     |29    |17     |1        |107              |127             |
|1       |2021-03-15 00:00:00|3    |11     |35    |23     |1        |115              |107             |
|1       |2021-03-22 00:00:00|3    |12     |30    |20     |1        |114              |115             |
|1       |2021-03-29 00:00:00|3    |13     |20    |31     |2        |111              |114             |
|1       |2021-04-05 00:00:00|4    |14     |40    |25     |1        |136              |111             |
|1       |2021-04-12 00:00:00|4    |15     |33    |22     |2        |129              |136             |
+--------+-------------------+-----+-------+------+-------+---------+-----------------+----------------+
only showing top 15 rows
val finalSnapshot: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 7 more fields]
```

Figure 2: Preprocessed dataset sample rows

3.5 Summary of Preprocessing Pipeline

The preprocessing pipeline progressed through four sequential stages, summarized in Table 5.

Table 5: Preprocessing pipeline summary

Stage	Input	Key Operation	Output
Cleaning	971,865 rows, 22 cols	Drop rows with missing geographic fields	952,563 rows, 22 cols
Integration	952,563 rows, 22 cols	Single-source schema documented, no joins	952,563 rows, 22 cols
Reduction	952,563 rows, 22 cols	Feature selection + pivoted weekly aggregation	4,644 rows, 33 cols
Transformation	4,644 rows, 33 cols	Temporal extraction, encoding, lag engineering	4,644 rows, 38 cols

The final preprocessed dataset of 4,644 rows and 38 columns was written to CSV and served as the unified input for all subsequent RDD, SQL, and machine learning phases.

The complete preprocessing code is located in 01_DataPreprocessing.scala in the submitted code folder.

4. RDD Operations

Seven RDD transformations and six RDD actions were implemented on the 952,563 cleaned incident-level records to uncover spatial, temporal, and behavioral crime patterns supporting the forecasting objective. Table 6 summarizes all operations implemented in this phase.

Table 6: RDD operations summary

#	Operation	Type	Purpose
T1	map	Transformation	Extract (district, 1) pairs
T2	filter	Transformation	Isolate THEFT records (22.1%)
T3	reduceByKey	Transformation	Aggregate total crimes per district
T4	flatMap	Transformation	Build crime-type frequency distribution
T5	groupByKey	Transformation	Count distinct crime types per district (28–30)
T6	sortByKey	Transformation	Rank districts in ascending order
T7	join	Transformation	Enrich with per-district arrest rates
A1–A6	count/take/reduce/first/foreach+collect/saveAsTextFile	Action	Verification, ranking, integrity check, persistence

4.1 RDD Design

All RDD operations were performed on the cleaned incident-level dataset (952,563 records) loaded as a Spark DataFrame and converted to an RDD of Row objects using .rdd. This approach was chosen deliberately over staying within the DataFrame/SQL API because RDDs provide fine-grained control over key-value pair operations[4] — particularly reduceByKey, groupByKey, and join — that are more naturally expressed at the RDD level for per-district aggregations.

The primary data model at the RDD level is RDD[(Int, Int)] representing (district_id, count) pairs, which enabled efficient distributed aggregation across all 23 districts. A secondary model RDD[(Int, (Int, Int))] was used after the join operation to carry (district_id, (total_crimes, arrests)) tuples for arrest rate computation. All RDD operations were performed before the weekly aggregation step to enable incident-level analyses — such as crime-type frequency distribution and per-district arrest rates — that are not possible on the aggregated weekly dataset used for machine learning.

4.2 Key Transformations and Actions

District Crime Volume Aggregation

Input: rawRDD — 952,563 Row objects loaded from the cleaned incident-level CSV, each representing one crime record with district and primary_type attributes.

Purpose: Compute total crime count per district to identify spatial inequality and establish the crime-level baseline the model must learn for each district.

Operations used: map (T1), reduceByKey (T3), sortBy, take (A2)

```
val districtOnePairs: RDD[(Int, Int)] = rawRDD.map { row =>
  val district = row.getAs[Int]("district")
  (district, 1)
}
val districtCrimeCount: RDD[(Int, Int)] =
  districtOnePairs.reduceByKey(_ + _)
val top5Districts = districtCrimeCount
  .sortBy(_._2, ascending = false).take(5)
```

Sample output:

```
District  8 -> 61,143 crimes
District  6 -> 58,262 crimes
District 12 -> 56,268 crimes
District  4 -> 54,444 crimes
District 11 -> 53,691 crimes
```

Insight: The top 5 districts account for approximately 30% of all 952,563 incidents despite representing fewer than 25% of active districts. District 8 leads with 61,143 crimes while District 31 records only 57 — a 1,072× difference. This extreme spatial inequality confirms the model must learn each district's individual crime level from historical records rather than applying a city-wide average.

Per-District Arrest Rate Enrichment

Input: rawRDD — 952,563 Row objects loaded from the cleaned incident-level CSV, each representing one crime record with district and primary_type attributes.

Purpose: Enrich district crime counts with arrest data using a join to measure enforcement effectiveness per district and validate the core motivation for proactive forecasting.

Operations used: filter, map, reduceByKey, join (T7), sortByKey (T6), saveAsTextFile (A6)

```
val districtArrestCount: RDD[(Int, Int)] = rawRDD
  .filter { row => row.getAs[Boolean]("arrest") }
  .map    { row => (row.getAs[Int]("district"), 1) }
  .reduceByKey(_ + _)
val districtEnriched = districtCrimeCount.join(districtArrestCount)
districtEnriched.sortByKey().map { case (district, (total, arrests)) =>
  val rate = arrests.toDouble / total * 100
  f"District $district%2d | Crimes: $total%,d | Rate: $rate%.2f%%"
}.saveAsTextFile("/Users/raghadfares/Desktop/BigData_Phase3/district_crime_summary")
```

Sample output (Districts 1–5):

```
District  1 | Crimes:  50,305 | Arrests:  8,036 | Rate: 16.0%
District  2 | Crimes:  47,391 | Arrests:  4,093 | Rate:  8.6%
District  3 | Crimes:  48,304 | Arrests:  5,127 | Rate: 10.6%
District  4 | Crimes:  54,444 | Arrests:  5,113 | Rate:  9.4%
District  5 | Crimes:  39,681 | Arrests:  5,437 | Rate: 13.7%
```

Insight: Arrest rates range from only 8.6% to 16.0% across all sampled districts. District 8 — the highest-crime district — has an arrest rate of just 10.6%, meaning 9 out of every 10 crimes there do not result in an arrest. This structurally limited enforcement effectiveness is the strongest operational justification for proactive crime forecasting.

4.3 Reflection on RDD Usage

RDDs made district-level key-value aggregations intuitive and the join straightforward. However, RDD code is more verbose than SQL—the arrest rate enrichment required three transformations plus a join versus a single SQL CTE—and type errors only surface at runtime. No caching or repartitioning was

applied given the manageable dataset size. In a production setting with multi-year data, caching rawRDD before repeated transformations would meaningfully reduce overhead.

5. SQL Operations

5.1 DataFrame and View Creation

Both datasets were loaded via spark.read (header, inferSchema) and registered as temporary views: clean_data.csv as crimes_cleaned and preprocessed_Data.csv as crimes_weekly, both available to all spark.sql() calls [7].

crimes_cleaned (952,563 rows, 22 cols) is used for incident-level queries (arrest status, crime type, hour). crimes_weekly (4,644 rows, 38 cols) is used for temporal trends and district averages. No join between them was performed as crimes_weekly is a derived aggregation of crimes_cleaned.

5.2 Key Analytical Queries

Eight non-trivial SQL queries were executed on two registered views covering aggregation, window functions, CTEs, statistical summaries, and subqueries. Table 7 provides an overview of all queries, followed by detailed writeups of five representative queries.
Table 7: SQL queries summary

Query	SQL Features	Key Result
Q1 – Monthly Trend	GROUP BY, AVG, SUM, MAX, MIN	Crime peaks July (avg 225.70/week); lowest February (184.25/week)
Q2 – Top 10 Crime Weeks	RANK() OVER, ORDER BY, LIMIT	District 1 holds top 4 weeks; highest = 674 crimes (July 2022)
Q3 – Arrest Rate	WITH (CTE), CASE WHEN, AVG	City avg = 13.28%; only 8 of 23 exceed it; District 8 = 10.59%
Q4 – Crime Type Stats	AVG, STDDEV, VARIANCE	District 8 leads avg total (292.03); District 1 most volatile
Q5 – Year-over-Year	LAG() OVER (PARTITION BY), CTE	District 1: +42.26% (2021→2022); Districts 4,6,11 peaked in 2023
Q6 – Peak Hours	HOUR(), SUM() OVER ()	Highest activity: 12:00–20:00; midnight spike is data artifact
Q7 – Domestic Crime	HAVING, conditional aggregation	16 of 23 districts exceed 15% domestic rate
Q8 – Above-Average	HAVING + subquery	11 of 23 districts exceed city avg (204.77); District 8 leads at 292.03

Q1: What is the total and average weekly crime count per month across all districts?

SQL features used: GROUP BY, SUM, AVG, MAX, MIN, COUNT, ORDER BY

```
SELECT month, COUNT(*) AS total_weeks,
  SUM(total_crime_count) AS total_crimes,
  ROUND(AVG(total_crime_count), 2) AS avg_weekly_crimes,
```

```
  MAX(total_crime_count) AS max_weekly_crimes,
  MIN(total_crime_count) AS min_weekly_crimes
FROM crimes_weekly
GROUP BY month ORDER BY month
```

Result excerpt:

month	total_crimes	avg_weekly_crimes	max_weekly_crimes
1	76,599	180.66	356
7	90,959	225.70	674
12	69,186	183.03	369

Interpretation: Crime peaks in July at an average of 225.70 crimes per week and drops to its lowest in February (184.25). This 25% seasonal swing directly validates the month and week_no features engineered in preprocessing — the model needs seasonal indicators to capture annual crime cycles.

Q2 : What are the 10 highest crime weeks on record and which district had them?

SQL features used: RANK() OVER, ORDER BY, LIMIT (window function)

```
SELECT district, week_start, month, week_no, total_crime_count,
  RANK() OVER (ORDER BY total_crime_count DESC) AS crime_rank
FROM crimes_weekly
ORDER BY total_crime_count DESC
LIMIT 10
```

Result excerpt:

district	week_start	total_crime_count	crimerank
1	2022-07-25	674	1
1	2023-07-31	539	2
1	2021-07-26	523	3
8	2024-09-09	409	5

Interpretation: District 1 holds the four highest crime weeks on record, all in July across consecutive years. The peak week (674 crimes, July 2022) is nearly three times the city average of 204.77. Eight of the top 10 weeks fall in July, September, or October, confirming that week_no and month are essential for the model to detect high-crime periods.

Q3: Which districts have an arrest rate above the city-wide average?

SQL features used: WITH (CTE), CASE WHEN, GROUP BY, AVG, subquery

```
WITH district_arrests AS (
  SELECT district,
    COUNT(*) AS total_crimes,
    ROUND(SUM(CASE WHEN arrest = true THEN 1 ELSE 0 END)
```

```
          * 100.0 / COUNT(*), 2) AS arrest_rate
  FROM crimes_cleaned GROUP BY district
),
city_average AS (
  SELECT ROUND(AVG(arrest_rate), 2) AS avg_arrest_rate
  FROM district_arrests
)
SELECT d.district, d.arrest_rate, c.avg_arrest_rate,
  CASE WHEN d.arrest_rate > c.avg_arrest_rate
       THEN 'ABOVE AVERAGE' ELSE 'BELOW AVERAGE' END AS performance
FROM district_arrests d, city_average c
ORDER BY d.arrest_rate DESC
```

Result excerpt:

district	arrest_rate	avg_arrest_rate	performance
11	26.07%	13.28%	ABOVE AVERAGE
1	15.97%	13.28%	ABOVE AVERAGE
8	10.59%	13.28%	BELOW AVERAGE
12	8.11%	13.28%	BELOW AVERAGE

Interpretation: Only 8 of 23 districts exceed the city average of 13.28%. District 8 — the highest-crime district — falls well below average at 10.59%, confirming that reactive enforcement is weakest precisely where forecasting is most needed.

Q5: How did total annual crime change per district between 2021 and 2024?

SQL features used: WITH (CTE), LAG() OVER (PARTITION BY district), CASE WHEN

```
WITH yearly_totals AS (
  SELECT district, year, COUNT(*) AS yearly_crime_count
  FROM crimes_cleaned GROUP BY district, year
),
yearly_with_change AS (
  SELECT district, year, yearly_crime_count,
    ROUND((yearly_crime_count - LAG(yearly_crime_count)
      OVER (PARTITION BY district ORDER BY year)) * 100.0
      / LAG(yearly_crime_count)
      OVER (PARTITION BY district ORDER BY year), 2) AS pct_change
  FROM yearly_totals
)
SELECT district, year, yearly_crime_count, pct_change,
  CASE WHEN pct_change > 0 THEN 'INCREASING'
       WHEN pct_change < 0 THEN 'DECREASING'
       ELSE 'NO CHANGE' END AS trend
FROM yearly_with_change
```

```
WHERE district IN (1, 4, 6, 8, 11, 12)
ORDER BY district, year
```

Result excerpt:

district	year	yearly_crimes	pct_change	trend
1	2022	12,609	+42.26%	INCREASING
8	2023	16,853	+15.43%	INCREASING
11	2024	13,561	−5.46%	DECREASING

Interpretation: Crime trends are non-linear and district-specific. District 1 surged 42.26% in a single year while District 11 peaked in 2023 then declined. This confirms that a constant-trend assumption would fail and validates the prev_week_total lag feature design.

Q8: Which districts consistently exceed the city-wide weekly crime average?

SQL features used: GROUP BY, HAVING, subquery in both SELECT and HAVING clause

```
SELECT district,
  ROUND(AVG(total_crime_count), 2) AS avg_weekly_crimes,
  MAX(total_crime_count) AS max_weekly_crimes,
  MIN(total_crime_count) AS min_weekly_crimes,
  ROUND((SELECT AVG(total_crime_count) FROM crimes_weekly), 2) AS
city_avg
FROM crimes_weekly
GROUP BY district
HAVING AVG(total_crime_count) > (
  SELECT AVG(total_crime_count) FROM crimes_weekly
)
ORDER BY avg_weekly_crimes DESC
```

Result excerpt:

district	avg_weekly_crimes	max_weekly	min_weekly	city_avg
8	292.03	409	76	204.77
6	278.20	364	71	204.77
1	240.47	674	63	204.77

Interpretation: Eleven of 23 districts consistently exceed the city average of 204.77. District 8 leads at 292.03 — 43% above average. District 1 has the widest range (63 to 674), confirming it as the most volatile district and the hardest to forecast accurately.

5.3 Summary of Insights from SQL

Five structural properties emerged from the eight SQL queries:

- Crime follows a strong seasonal pattern peaking in July (avg 225.70 crimes/week) and dropping in February (184.25/week), confirming that month and week_no are necessary features for the forecasting model to capture annual cycles.
- The highest recorded week (District 1, July 2022, 674 crimes) is nearly three times the city average, confirming that spike detection requires a lag feature — the model must see recent momentum to anticipate extreme weeks.
- The city-wide arrest rate is only 13.28%, with 15 of 23 districts falling below this threshold. District 8, the highest-crime district, has only a 10.59% arrest rate. This structural limitation of reactive policing is the strongest operational justification for the forecasting system.
- Each district has a distinct behavioral crime-type profile (Districts 12, 18, 19 are THEFT-dominated; Districts 6, 4, 11 are BATTERY-dominated), confirming that treating districts as unique categorical entities rather than interchangeable numeric values was the correct design decision.
- Eleven of 23 districts consistently exceed the city weekly average of 204.77 crimes, with District 8 leading at 292.03. These 11 districts represent the highest forecasting priority where prediction accuracy has the greatest operational impact.

6. Machine Learning

6.1 Problem Formulation

The task is regression: predicting total_crime_count—weekly crime incidents per district—ranging from 1 to 674 (mean: 204.77, SD: 66.86).

Two configurations were evaluated: full (35 features, including 31 crime-type pivot columns) and reduced (4 features: month, week_no, district_indexed, prev_week_total). The full set contains a mathematical identity—the crime-type columns sum to the target—making the reduced configuration the operationally valid evaluation.

The primary challenge is high week-to-week variance, especially in District 1 (variance=5,088.88), creating inherently difficult prediction periods.

Table 8: ML problem formulation

Element	Description
Target variable	total_crime_count — total weekly crimes per district
Algorithms	Linear Regression, Decision Tree Regressor, Random Forest Regressor
Baseline	Mean prediction — predicts training mean for every test row

Element	Description
Evaluation	RMSE, MAE, R^2 on 30% held-out test set
Train/test split	70/30, seed = 42 — split BEFORE scaling to prevent data leakage
Feature configurations	Full: 35 features | Reduced: 4 features

6.2 Feature Pipeline

The Spark ML pipeline has five stages[6]:

Stage 1: Null filling: na.fill(0) handles sparse low-frequency crime-type columns.

Stage 2 : VectorAssembler: Selected features combined into features_raw (35 cols for full; 4 for reduced).

Stage 3: Train/test split: 70/30 (seed=42), producing 3,312 train / 1,332 test rows, applied before scaling to prevent leakage.

Stage 4: StandardScaler: Fitted on training data only, then applied to both sets to prevent test information leakage.

Stage 5 : Model training and evaluation on the scaled datasets.

na.fill(0) → VectorAssembler → randomSplit (70/30) → StandardScaler (fit on train only) → Model Training → Evaluation

```
    | println(f"${"RF — Reduced Features"}%-35s ${rfReducedRMSE}%10.4f ${rfReducedMAE}%10.4f ${rfReducedR2}%10.4f")
    | println("=" * 75)
LR Reduced RMSE: 29.045191996284945
LR Reduced MAE:  20.984033174664535
LR Reduced R2:   0.8207910263279232
DT Reduced RMSE: 29.18853658401931
DT Reduced MAE:  21.349374361299244
DT Reduced R2:   0.8198177878472602
RF Reduced RMSE: 28.542894369426964
RF Reduced MAE:  21.279497302127364
RF Reduced R2:   0.826935787142312

===========================================================================
Model                                RMSE       MAE        R2
---------------------------------------------------------------------------
Baseline (Mean)                    68.6117    54.4285    -0.0000
--- Full Feature Set (35 features) ---
LR — Full Features                  0.0896     0.0611     1.0000
DT — Full Features                 28.4365    20.6533     0.8282
RF — Full Features                 21.0664    14.8280     0.9057
--- Reduced Feature Set (4 features) ---
LR — Reduced Features              29.0452    20.9840     0.8208
DT — Reduced Features              29.1885    21.3494     0.8190
RF — Reduced Features              28.5429    21.2795     0.8269
===========================================================================
val lrReduced: org.apache.spark.ml.regression.LinearRegression = linReg_fc30cbeda999
val lrReducedModel: org.apache.spark.ml.regression.LinearRegressionModel = LinearRegressionModel: uid=linReg_fc30cbeda999, numFeatures=4
val lrReducedPred: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 39 more fields]
val lrReducedRMSE: Double = 29.045191996284945
val lrReducedMAE: Double = 20.984033174664535
val lrReducedR2: Double = 0.8207910263279232
val dtReduced: org.apache.spark.ml.regression.DecisionTreeRegressor = dtr_b8aca5e6d5a9
val dtReducedModel: org.apache.spark...
```

Figure 3: Full model comparison

6.3 Model Choice

Three algorithms span a range of complexity. Linear Regression serves as a statistical benchmark with interpretable coefficients. Decision Tree captures non-linear relationships and handles categorical district data without scaling. Random Forest—the primary candidate—averages 50 trees to reduce overfitting[5], critical for a high-variance dataset.

Hyperparameters: LR used L2 regularization (regParam=0.1, 100 iterations); DT and RF used max depth=5; RF used 50 trees (seed=42). Shallow settings prevent overfitting on 3,312 training rows.

6.4 Training and Evaluation Setup

The dataset was split 70/30 using randomSplit(Array(0.7, 0.3), seed = 42), producing 3,312 training rows and 1,332 test rows. The fixed seed ensures full reproducibility. Since this is a regression task there is no class imbalance concern and no resampling was required.

Three complementary evaluation metrics were selected. RMSE measures average prediction error in original units of crimes per week, making it directly interpretable for patrol planning. MAE provides a measure less sensitive to occasional large errors. R^2 measures the proportion of weekly crime variance explained by the model, where 0 represents always predicting the mean and 1 represents perfect prediction. All metrics were computed on the held-out test set only.

6.5 Results

Table 9: Full model comparison across all configurations

Model	RMSE	MAE	R^2	Notes
Baseline (Mean)	68.61	54.43	≈0.00	Performance floor
— Full Feature Set (35 features) —				
LR – Full*	0.09	0.06	0.9999983	Identity artifact
DT – Full	28.44	20.65	0.8282	Genuine learning
RF – Full	21.07	14.83	0.9057	Best full-feature model
— Reduced Feature Set (4 features) —				
LR – Reduced	29.05	20.98	0.8208	Honest forecast
DT – Reduced	29.19	21.35	0.8190	Honest forecast
RF – Reduced	28.54	21.28	0.8269	PRIMARY RESULT

6.6 Interpretation

Feature Importance

Table 10: Reduced RF feature importances

Rank	Feature	Importance	Interpretation
1	prev_week_total	82.9%	Last week's count is the dominant predictor
2	district_indexed	12.7%	Each district has a unique crime-level baseline
3	week_no	3.0%	Within-year weekly cycles

Rank	Feature	Importance	Interpretation
4	month	1.3%	Seasonal monthly patterns

Model Interpretation

The reduced Random Forest ($R^2 = 0.827$, RMSE = 28.54) is the primary result. Using only 4 features, the model predicts weekly district-level crime counts within approximately 28–29 crimes on average — a 58.4% reduction in error compared to the mean baseline. In practical terms: if a district records 200 crimes in a week, the model would typically predict between 172 and 228.

The convergence of all three reduced-feature models to similar R^2 (0.819–0.827) confirms the 4-feature set captures most available forecasting signal. prev_week_total drives 82.9% of decisions, confirming historical momentum is the strongest short-term forecasting signal. The full RF ($R^2 = 0.906$) uses same-week crime-type data unavailable before the week ends, making the reduced results the operationally valid evaluation.

Error Analysis

The reduced Random Forest achieves consistent performance across most districts, but prediction accuracy varies with district volatility. District 1 has the highest week-to-week variance in the dataset (variance = 5,088.88 as confirmed in Q4 of the SQL phase), with weekly counts ranging from a minimum of 63 to a maximum of 674. In high-variance weeks — particularly July peaks — the model tends to underpredict because the lag feature captures the previous week's count, which may not reflect the full magnitude of an upcoming spike. This is an inherent limitation of single-step lag features: they capture momentum but cannot predict sudden surges driven by external events such as summer festivals or heatwaves.

Districts with more stable weekly patterns, such as District 3 (variance = 1,279.59) and District 6 (variance = 1,305.87), are predicted more accurately. The three reduced-feature models converge to nearly identical R^2 values (0.819–0.827), suggesting the performance ceiling is driven by the available information rather than model complexity — adding more sophisticated models is unlikely to improve results significantly without incorporating external contextual features.

6.7 Limitations

Three methodological limitations affect the current results. First, the feature set is restricted to internal temporal, spatial, and lag signals. External variables such as weather conditions, school calendars, public events, and economic indicators that are known to influence crime activity are not included. Incorporating these could potentially push R^2 beyond the current ceiling of approximately 0.83. Second, the random 70/30 train/test split mixes records from different years (2021–2024) rather than using a strict temporal boundary. A more rigorous evaluation would train on 2021–2023 and test exclusively on 2024, ensuring the model is evaluated on genuinely future data it has never seen. The current split may slightly overestimate generalization performance because the model can learn from 2024 patterns that happen to fall in the training set. Third, the current pipeline does not include hyperparameter tuning — parameters

such as tree depth and regularization strength were set manually based on the dataset size. A cross-validated grid search could identify better configurations, though the small dataset size limits the practical benefit.

7. Conclusion and Future Work

7.1 Summary of Contributions

This project developed and validated a complete big data analytics pipeline for district-level weekly crime count forecasting in Chicago using Apache Spark with Scala. Across five analytical phases, the team cleaned and aggregated 971,865 raw incident records into 4,644 structured weekly rows, explored spatial and behavioral crime patterns using 7 RDD operations and 8 SQL queries, and trained and evaluated three regression models under two feature configurations.

The four key findings of this project are: (1) weekly crime counts per district can be predicted with R^2 = 0.827 and RMSE = 28.54 using only four features available before a week begins, making the system practically deployable; (2) the previous week's total crimes (prev_week_total) is the single strongest predictor, accounting for 82.9% of the model's decisions and validating the lag feature engineering decision; (3) city-wide arrest rates average only 13.28%, confirming that reactive policing is structurally limited and proactive forecasting is most valuable in the highest-burden districts; (4) crime is highly spatially concentrated — the top 5 districts account for 30% of all incidents — making accurate district-level forecasts operationally critical for resource allocation.

7.2 Reflection

The project covered the full Spark ecosystem from ingestion through MLlib. The most significant challenge was preventing data leakage in the StandardScaler step, requiring careful split-before-scale ordering.

A key insight was discovering the mathematical identity in the full feature set—crime-type columns summing to the target inflated LR performance without genuine value. This required a reduced-feature evaluation and demonstrated that strong metrics alone do not confirm genuine learning.

7.3 Future Work

Concrete next steps for extending this work include: incorporating external features such as weather data, school calendars, and public event schedules that are known to drive short-term crime fluctuations; applying a strict temporal train/test split (train on 2021–2023, test on 2024) for a more rigorous out-of-sample evaluation; experimenting with Gradient Boosted Trees, which may better capture the non-linear spike behavior observed in high-variance districts such as District 1; extending the forecasting system to predict individual crime-type volumes separately rather than total counts, enabling more targeted patrol deployment; and developing a real-time weekly forecasting dashboard that ingests new data automatically and updates predictions each Monday before the week begins.

8. References

[1] W. L. Perry, B. McInnis, C. C. Price, S. C. Smith, and J. S. Hollywood, *Predictive Policing: The Role of Crime Forecasting in Law Enforcement Operations*, RAND Corporation, 2013. [Online]. Available: https://doi.org/10.7249/RR233. [Accessed: 10-May-2026].

[2] Chicago Police Department, "Workforce Allocation Study: Executive Summary," Office of Constitutional Policing and Reform, 2024. [Online]. Available: https://www.chicagopolice.org/wp-content/uploads/WFA_ES.pdf. [Accessed: 10-May-2026].

[3] City of Chicago, "Crimes - 2001 to Present," Chicago Data Portal, 2026. [Online]. Available: https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2. [Accessed: 10-May-2026].

[4] B. Chambers and M. Zaharia, *Spark: The Definitive Guide*, O'Reilly Media, 2018. [Online]. Available: https://analyticsdata24.wordpress.com/wp-content/uploads/2020/02/spark-the-definitive-guide40www.bigdatabugs.com_.pdf. [Accessed: 26-Feb-2026].

[5] J. S. Damji, B. Wenig, T. Das, and D. Lee, *Learning Spark: Lightning-Fast Data Analytics*, 2nd ed., O'Reilly Media, 2020. [Online]. Available: https://vdoc.pub/download/learning-spark-lightning-fast-data-analytics-61bd73j55np0. [Accessed: 15-Mar-2026].

[6] Apache Software Foundation, "Spark MLlib Guide," Apache Spark Documentation, 2026. [Online]. Available: https://spark.apache.org/docs/latest/ml-guide.html. [Accessed: 16-Mar-2026].

[7] Apache Software Foundation, "Spark SQL Built-in Functions," Apache Spark Documentation, 2026. [Online]. Available: https://spark.apache.org/docs/latest/api/sql/index.html. [Accessed: 15-Mar-2026].

[8] TutorialsPoint, "Apache Spark Tutorial," TutorialsPoint (I) Pvt. Ltd., 2015. [Online]. Available: https://www.tutorialspoint.com/apache_spark/apache_spark_tutorial.pdf. [Accessed: 13-Mar-2026].

[9] City of Chicago, "Boundaries – Police Districts (current)," Chicago Open Data Portal. [Online]. Available: https://data.cityofchicago.org/Public-Safety/Boundaries-Police-Districts-current-/fthy-xz3r. [Accessed: 27-Feb-2026].